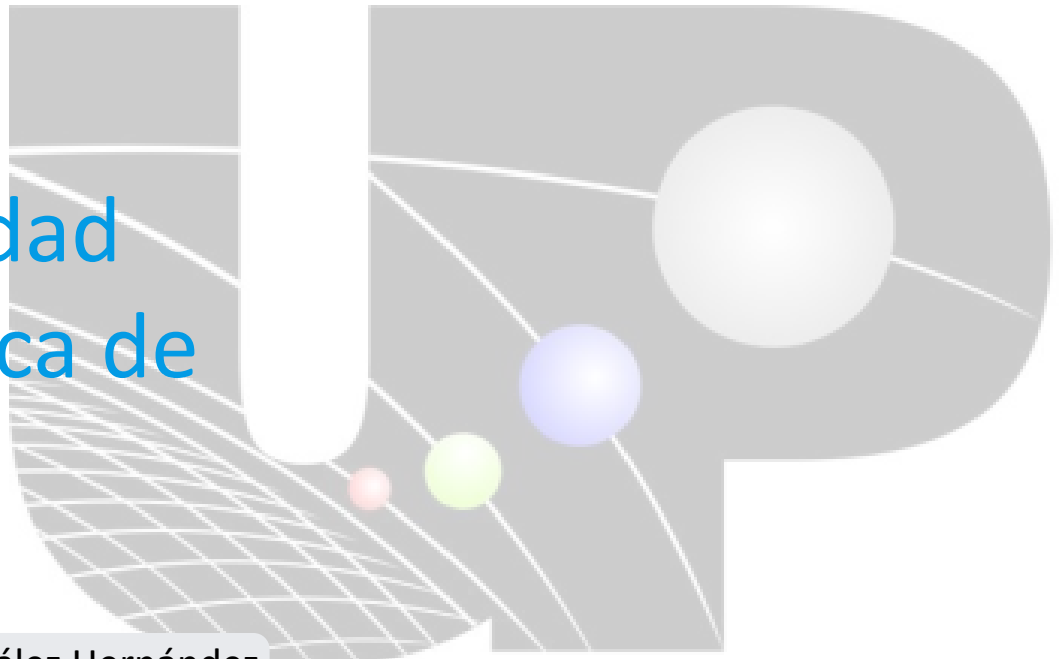


Universidad Politécnica de Chiapas



Edwar Daniel González Hernández

243684

Cesar Yair Toledo Villarreal

243739

Carlos Eduardo Lopez Ochoa

243717

Grado y Grupo: 5-C

Examen 1

Base de datos Avanzada

1. ¿Por qué escogimos este caso (Caso 3-B)?

Escogimos el caso de **Gestión de Eventos Corporativos** por tres razones fundamentales que nos permitieron explorar a fondo el diseño de bases de datos relacionales:

1. **Complejidad de Relaciones N:M con Atributos:** A diferencia de casos más simples, este escenario presentaba el reto de gestionar relaciones "muchos a muchos" que no solo unen tablas, sino que **guardan estado**. El hecho de que un ponente (Dra. Vega) tenga roles distintos según la sesión, o que un inscrito pueda no asistir, nos obligó a diseñar tablas intermedias ricas en información (**Participacion** con rol, **Inscripcion** con asistencia real), superando el modelo básico de relación simple.
2. **Modelado Financiero y Transaccional:** El requerimiento de manejar "pagos parciales" y "reembolsos" nos pareció el desafío más cercano a un sistema ERP real. Nos permitió demostrar cómo separar la **deuda** (Paquete) del **flujo de efectivo** (Pagos), evitando errores comunes de diseño donde se sobrescriben los montos en una sola columna.
3. **Lógica de Negocio vs. Lógica Física:** El caso distingue entre la capacidad física de un salón (infraestructura) y el cupo lógico de un taller (regla de negocio). Queríamos diseñar un sistema flexible capaz de manejar estas restricciones duales sin depender enteramente del software frontend.

2. ¿Por qué lo diseñamos así?

Nuestro diseño definitivo se fundamenta en la **Integridad de Datos** y la **Escalabilidad**, priorizando la **3ra Forma Normal (3FN)**. Justificamos nuestras decisiones arquitectónicas clave de la siguiente manera:

1. **Atomización Financiera (Relación 1:N Paquete-Pagos):** Decidimos **no** guardar el "monto pagado" como un campo acumulativo en la tabla **Paquete**. En su lugar, creamos una entidad **Pago** separada.
 - *Justificación:* Esto garantiza la trazabilidad. Si una empresa paga en dos exhibiciones (como Grupo Oriente), el sistema guarda la fecha y monto exacto de cada transacción. Además, permite manejar estados individuales (ej. un pago "Reembolsado" y otro "Pagado") sin perder el historial financiero.
2. **Normalización de Entidades Externas (Empresas y Patrocinios):** A diferencia de un diseño plano, vinculamos la tabla **Patrocinio** con la tabla **Empresa** mediante una llave foránea (**id_empresa**), en lugar de escribir el nombre de la empresa como texto.
 - *Justificación:* Esto nos permite generar reportes consolidados de "Inversión Total por Cliente" (sumando Paquetes + Patrocinios) de manera eficiente mediante SQL. Si hubiéramos usado texto, un error de dedo ("Innova Tech" vs "InnovaTech") rompería los reportes financieros.

3. **Tablas Puente con Atributos de Contexto:** En las tablas intermedias [Participacion_Ponente](#) e [Inscripcion](#), añadimos columnas clave ([rol](#) y [asistio_real](#)).
 - *Justificación:* Entendimos que el "Rol" no es inherente a la persona, sino al contexto del evento. Un diseño rígido hubiera impedido que la Dra. Vega fuera panelista y ponente el mismo día. Del mismo modo, el booleano [asistio_real](#) es la única forma de calcular tasas de ausentismo (No Show) sin eliminar el registro histórico de que la persona tuvo intención de ir.
4. **Restricciones a Nivel de Base de Datos:** Implementamos [CHECK constraints](#) para fechas y montos positivos directamente en el esquema.
 - *Justificación:* Adoptamos la filosofía de "Defense in Depth" (Defensa en Profundidad). Aunque la aplicación web debe validar datos, la base de datos es la última barrera para evitar corrupción financiera (ej. saldos negativos) o inconsistencias temporales (eventos que terminan antes de empezar).

2) RESTRICCIONES E INTEGRIDAD DE DATOS

Para garantizar la calidad de la información y cumplir con las reglas de negocio del caso "Gestión de Eventos Corporativos", se han identificado e implementado las siguientes restricciones en el esquema de base de datos.

A. Restricciones de Dominio (CHECK Constraints) Garantizan que los datos numéricos y de fechas tengan sentido lógico.

Tabla	Campo(s)	Regla (SQL Check)	Justificación para la Defensa
Evento	fecha_inicio, fecha_fin	CHECK (fecha_fin >= fecha_inicio)	Previene inconsistencias cronológicas; un evento no puede terminar antes de comenzar.
Sesión	fecha_hora_inicio, fecha_hora_fin	CHECK (fecha_hora_fin > fecha_hora_inicio)	Garantiza que las sesiones (charlas, talleres) tengan una duración positiva válida.
Pago / Patrocinio	monto	CHECK (monto >= 0)	Impide errores financieros como pagos negativos. Los reembolsos se manejan vía estado, no con signo negativo.
Paquete	lugares_totales	CHECK (lugares_totales > 0)	Un paquete corporativo debe incluir al menos un lugar para ser válido.

Paquete	costo_total	CHECK (costo_total >= 0)	El costo de un paquete no puede ser negativo.
---------	-------------	--------------------------	---

Restricciones de Estado (Enumeraciones) Limitan los valores de texto a una lista controlada por el negocio.

Tabla	Campo	Valores Permitidos	Justificación
Patrocinio	nivel	Oro', 'Plata', 'Bronce'	Estrictamente definido en el caso para clasificar patrocinadores ("InnovaTech" nivel Oro, "Café Digital" nivel Bronce).
Paquete	estado	Activo', 'Cancelado'	Necesario para gestionar el caso de "Soluciones del Pacífico", que canceló su paquete después de pagar.
Pago	estado	Pagado', 'Reembolsado'	Permite rastrear el dinero devuelto sin borrar el registro histórico del pago (auditoría).
Sesión	tipo	Charla', 'Panel', 'Taller'	Clasifica las actividades según la descripción del evento "Tech Summit Sureste".

Restricciones de Unicidad (UNIQUE) Evitan la duplicidad de información crítica.

1. Identidad Fiscal Única:

- **Restricción:** **UNIQUE (rfc)** en la tabla [Empresa](#).
- **Justificación:** El RFC (ej. GOR180901MNO) es el identificador legal único. El sistema no debe permitir dos empresas con el mismo RFC.

2. No Doble Inscripción:

- **Restricción:** **UNIQUE (id_asistente, id_sesion)** en la tabla [Inscripcion](#).
- **Justificación:** Un asistente físico (Ana) no puede ocupar dos lugares en la misma sesión ("Taller de Datos"). Esto es vital para controlar el cupo real.

3. Unicidad de Rol por Sesión:

- **Restricción:** **UNIQUE (id_ponente, id_sesion)** en tabla [Participacion_Ponente](#).
- **Justificación:** Aunque un ponente puede estar en múltiples sesiones, dentro de una misma sesión solo debe tener un registro de rol asignado para evitar redundancia.

3) Queries SQL de ejemplo:

SELECT con JOINS, agregaciones, GROUP BY, HAVING, CASE/COALESCE.

1. Agenda Detallada con Ponentes y Roles

Esta consulta recupera la agenda completa del evento "Tech Summit Sureste 2026". Muestra el nombre de la sesión, en qué salón ocurre, el horario definido, y lista a todos los ponentes involucrados, especificando qué rol juega cada uno (si es presentador, moderador, etc.) en esa sesión en particular.

SQL

SELECT

e.nombre AS nombre_evento,

s.titulo AS sesion,

sa.nombre AS salon,

CONCAT(s.fecha_hora_inicio, ' - ', s.fecha_hora_fin) AS horario,

p.nombre AS ponente,

pp.rol

FROM Sesion s

JOIN Evento e ON s.id_evento = e.id_evento

JOIN Salon sa ON s.id_salon = sa.id_salon

JOIN Participacion_Ponente pp ON s.id_sesion = pp.id_sesion

JOIN Ponente p ON pp.id_ponente = p.id_ponente

WHERE e.nombre = 'Tech Summit Sureste 2026'

ORDER BY s.fecha_hora_inicio;

2. Total de Asistentes por Empresa

Esta consulta cuenta cuántos asistentes físicos envió cada empresa cliente al evento. Sirve para validar si las empresas están utilizando todos los lugares de los paquetes que compraron.

SQL

SELECT

emp.razon_social,

COUNT(a.id_asistente) AS total_asistentes_enviados

FROM Empresa emp

JOIN Paquete p ON emp.id_empresa = p.id_empresa

JOIN Asistente a ON p.id_paquete = a.id_paquete

JOIN Evento evt ON p.id_evento = evt.id_evento

WHERE evt.nombre = 'Tech Summit Sureste 2026'

GROUP BY emp.id_empresa, emp.razon_social;

3. Ponentes con Múltiples Roles (Caso Dra. Vega)

Identifica a los ponentes que son versátiles dentro de un mismo evento. Filtra y muestra solo a aquellos que desempeñaron más de un rol distinto (por ejemplo, alguien que fue Ponente en una charla y Panelista en otra), ignorando a quienes solo tuvieron un tipo de participación.

SQL

SELECT

pon.nombre,

COUNT(DISTINCT pp.rol) AS roles_distintos

FROM Ponente pon

JOIN Participacion_Ponente pp ON pon.id_ponente = pp.id_ponente

JOIN Sesion s ON pp.id_sesion = s.id_sesion

JOIN Evento e ON s.id_evento = e.id_evento

WHERE e.nombre = 'Tech Summit Sureste 2026'



```
GROUP BY pon.id_ponente, pon.nombre
```

```
HAVING COUNT(DISTINCT pp.rol) > 1;
```

4. Ingreso Total por Tipo de Paquete

Calcula cuánto dinero ha ingresado realmente a la empresa organizadora, desglosado por el tipo de paquete vendido (Premium, Básico, etc.). Considera únicamente los pagos que ya tienen el estado "Pagado", excluyendo deudas o reembolsos.

SQL

SELECT

 pq.tipo_paquete,

 SUM(pg.monto) AS ingreso_total_cobrado

FROM Paquete pq

JOIN Pago pg ON pq.id_paquete = pg.id_paquete

WHERE pg.estado = 'Pagado'

GROUP BY pq.tipo_paquete

ORDER BY ingreso_total_cobrado DESC;

5. Asistentes con Alta Participación

Busca a los asistentes "intensos" del evento. Muestra el nombre y la empresa de origen de aquellas personas que se inscribieron a más de 3 sesiones diferentes dentro del mismo evento.

SQL

SELECT

 a.nombre AS asistente,

 emp.razon_social AS viene_de_empresa,

 COUNT(i.id_sesion) AS sesiones_inscritas

```
FROM Asistente a

JOIN Inscripcion i ON a.id_asistente = i.id_asistente

JOIN Sesion s ON i.id_sesion = s.id_sesion

JOIN Evento e ON s.id_evento = e.id_evento

JOIN Paquete p ON a.id_paquete = p.id_paquete

JOIN Empresa emp ON p.id_empresa = emp.id_empresa

WHERE e.nombre = 'Tech Summit Sureste 2026'

GROUP BY a.id_asistente, a.nombre, emp.razon_social

HAVING COUNT(i.id_sesion) > 3;
```

6. Recaudación por Patrocinios (Año Actual)

Muestra el total de dinero recaudado exclusivamente por concepto de patrocinios durante el año 2026, agrupando los montos según el nivel del patrocinio (Oro, Plata, Bronce).

SQL

```
SELECT

    pt.nivel,

    SUM(pt.monto) AS total_recaudado

FROM Patrocinio pt

JOIN Evento e ON pt.id_evento = e.id_evento

WHERE e.fecha_inicio >= '2026-01-01' AND e.fecha_fin <= '2026-12-31'

GROUP BY pt.nivel;
```

7. Sesiones "Sold Out" (Llenas)

Detecta las sesiones exitosas que alcanzaron su capacidad máxima. Compara el número de inscripciones registradas contra el cupo máximo definido para la sesión, y muestra solo aquellas donde las inscripciones igualan o superan dicho límite.



SQL

SELECT

s.titulo,

s.cupo_maximo,

COUNT(i.id_inscripcion) AS inscritos_totales

FROM Sesion s

LEFT JOIN Inscripcion i ON s.id_sesion = i.id_sesion

GROUP BY s.id_sesion, s.titulo, s.cupo_maximo

HAVING COUNT(i.id_inscripcion) >= s.cupo_maximo;

4) Reportes Implementados (Vistas)

1. Resumen Financiero por Evento ([v_resumen_financiero](#))

Este reporte genera una vista consolidada de los ingresos de cada evento. La lógica aplicada es la siguiente:

1. Calcula la suma de los costos de todos los **Paquetes Activos** vendidos para el evento. Si no hay paquetes, asume 0.
2. Calcula la suma de todos los montos de **Patrocinios** recibidos. Si no hay patrocinadores, asume 0.
3. Suma ambos conceptos para obtener el **Ingreso Total Global**.
4. Calcula qué **porcentaje** del ingreso total proviene exclusivamente de los patrocinios.
5. Se incluye una validación para evitar errores de división entre cero en caso de que un evento no tenga ingresos aún.

Código SQL:

SQL

```
CREATE VIEW v_resumen_financiero AS
```

```
SELECT
```

```
    e.nombre AS evento,
```

```
    -- Ingresos por Paquetes (convertimos nulos a 0)
```

```
    COALESCE(SUM(paq.costo_total), 0) AS ingresos_paquetes,
```

```
    -- Ingresos por Patrocinios (subconsulta para evitar duplicidad en JOINs)
```

```
    COALESCE(pat.total_patrocinios, 0) AS ingresos_patrocinios,
```

```
    -- Ingreso Total
```

```
    (COALESCE(SUM(paq.costo_total), 0) + COALESCE(pat.total_patrocinios, 0)) AS ingreso_total_global,
```

```
    -- Cálculo de Porcentaje de Aportación de Patrocinios
```

```
CASE
```

```
    WHEN (COALESCE(SUM(paq.costo_total), 0) + COALESCE(pat.total_patrocinios, 0)) = 0 THEN 0
```

```
ELSE (COALESCE(pat.total_patrocinios, 0) * 100.0 / (COALESCE(SUM(paq.costo_total), 0) +  
COALESCE(pat.total_patrocinios, 0)))  
  
END AS porcentaje_patrocinios  
  
FROM Evento e  
  
LEFT JOIN Paquete paq ON e.id_evento = paq.id_evento AND paq.estado = 'Activo'  
  
LEFT JOIN (  
  
    SELECT id_evento, SUM(monto) as total_patrocinios  
  
    FROM Patrocinio  
  
    GROUP BY id_evento  
  
) pat ON e.id_evento = pat.id_evento  
  
GROUP BY e.id_evento, e.nombre, pat.total_patrocinios;
```

2. Asistencia y Ocupación por Sesión ([v_asistencia_sesion](#))

Este reporte mide el éxito operativo de cada sesión (charlas, talleres). La lógica es:

1. Cuenta el total de personas que se registraron en la base de datos (**Inscritos**).
2. Cuenta cuántas de esas personas tienen la marca de verificación de asistencia física (**Asistentes Reales**).
3. Calcula la **Tasa de Asistencia** (Asistentes Reales / Total Inscritos).
4. Clasifica la ocupación comparando los inscritos contra el **Cupo Máximo** de la sesión bajo las siguientes reglas:
 - Si los inscritos igualan o superan el cupo: "Lleno Total".
 - Si superan el 80%: "Alta Ocupación".
 - Si superan el 50%: "Media Ocupación".
 - Menos del 50%: "Baja Ocupación".

Código SQL:

SQL

```
CREATE VIEW v_asistencia_sesion AS
```

```
SELECT
```

```
s.titulo AS sesion,

sal.nombre AS salon,

s.cupo_maximo,

COUNT(i.id_inscripcion) AS total_inscritos,

-- Suma condicional para contar solo los que sí asistieron

SUM(CASE WHEN i.asistio_real = true THEN 1 ELSE 0 END) AS asistentes_reales,

-- Tasa de asistencia (convertida a porcentaje)

CASE

    WHEN COUNT(i.id_inscripcion) = 0 THEN 0

    ELSE (SUM(CASE WHEN i.asistio_real = true THEN 1 ELSE 0 END) * 100.0 / COUNT(i.id_inscripcion))

END AS tasa_asistencia_pct,

-- Clasificación cualitativa de ocupación

CASE

    WHEN COUNT(i.id_inscripcion) >= s.cupo_maximo THEN 'Lleno Total'

    WHEN COUNT(i.id_inscripcion) >= (s.cupo_maximo * 0.8) THEN 'Alta Ocupación'

    WHEN COUNT(i.id_inscripcion) >= (s.cupo_maximo * 0.5) THEN 'Media Ocupación'

    ELSE 'Baja Ocupación'

END AS estatus_ocupacion

FROM Sesion s

JOIN Salon sal ON s.id_salon = sal.id_salon

LEFT JOIN Inscripcion i ON s.id_sesion = i.id_sesion

GROUP BY s.id_sesion, s.titulo, sal.nombre, s.cupo_maximo;
```

3. Inversión y Retorno por Empresa Cliente ([v_inversion_empresa](#))

Este reporte permite a las empresas cliente ver el retorno de su inversión. La lógica incluye:

1. Contar cuántos paquetes distintos ha comprado la empresa.
2. Sumar el monto de los **Pagos** que realmente se han realizado (estado 'Pagado'), ignorando el costo teórico del paquete para reflejar el flujo de efectivo real.
3. Contar el número de asistentes únicos que la empresa envió al evento.
4. Calcular el **Costo Promedio por Asistente** dividiendo lo pagado entre el número de asistentes enviados.

Código SQL:

SQL

```
CREATE VIEW v_inversion_empresa AS
```

```
SELECT
```

```
emp.razon_social,
```

```
COUNT(DISTINCT paq.id_paquete) AS paquetes_comprados,
```

```
-- Suma de pagos reales registrados en el sistema
```

```
COALESCE(SUM(pg.monto), 0) AS total_pagado_real,
```

```
COUNT(DISTINCT a.id_asistente) AS asistentes_enviados,
```

```
-- Costo promedio: Cuánto le costó a la empresa cada persona que envió
```

```
CASE
```

```
    WHEN COUNT(DISTINCT a.id_asistente) = 0 THEN 0
```

```
    ELSE COALESCE(SUM(pg.monto), 0) / COUNT(DISTINCT a.id_asistente)
```

```
END AS costo_promedio_asistente
```

```
FROM Empresa emp
```

```
JOIN Paquete paq ON emp.id_empresa = paq.id_empresa
```

```
LEFT JOIN Pago pg ON paq.id_paquete = pg.id_paquete AND pg.estado = 'Pagado'
```

```
LEFT JOIN Asistente a ON paq.id_paquete = a.id_paquete
```

```
GROUP BY emp.id_empresa, emp.razon_social;
```

4. Balance de Pagos y Deudas ([v_balance_pagos](#))

Este reporte es una herramienta de cobranza para identificar saldos pendientes. La lógica es:

1. Selecciona únicamente los Paquetes con estado "Activo".
2. Muestra el costo total pactado del paquete.
3. Suma todos los pagos parciales asociados a ese paquete.
4. Calcula el **Saldo Pendiente** restando lo pagado al costo total.
5. Genera una etiqueta de estado ("Sin Pagos", "Pago Parcial", "Liquidado") para facilitar la lectura administrativa.

Código SQL:

SQL

```
CREATE VIEW v_balance_pagos AS
```

```
SELECT
```

```
emp.razon_social,
```

```
paq.tipo_paquete,
```

```
paq.costo_total AS costo_paquete,
```

```
COALESCE(SUM(pg.monto), 0) AS monto_pagado,
```

```
-- Cálculo de deuda remanente
```

```
(paq.costo_total - COALESCE(SUM(pg.monto), 0)) AS saldo_pendiente,
```

```
-- Estado de la deuda
```

```
CASE
```

```
    WHEN COALESCE(SUM(pg.monto), 0) = 0 THEN 'Sin Pagos'
```

```
    WHEN COALESCE(SUM(pg.monto), 0) < paq.costo_total THEN 'Pago Parcial'
```

```
    WHEN COALESCE(SUM(pg.monto), 0) >= paq.costo_total THEN 'Liquidado'
```

```
END AS estatus_deuda
```

```
FROM Paquete paq
```

```
JOIN Empresa emp ON paq.id_empresa = emp.id_empresa
```

```
LEFT JOIN Pago pg ON paq.id_paquete = pg.id_paquete AND pg.estado = 'Pagado'
```

```
WHERE paq.estado = 'Activo'
```

```
GROUP BY paq.id_paquete, emp.razon_social, paq.tipo_paquete, paq.costo_total;
```

5. Actividad de Ponentes ([v_ponentes_activos](#))

Identifica a los ponentes clave del evento. La lógica consiste en:

1. Agrupar los registros por Ponente.
2. Contar en cuántas sesiones distintas participa (volumen de actividad).
3. Contar en cuántos eventos diferentes ha estado (fidelidad).
4. Listar los roles que ha desempeñado (Presentador, Panelista, etc.) para entender su perfil.

Código SQL:

SQL

```
CREATE VIEW v_ponentes_activos AS
```

```
SELECT
```

```
    p.nombre AS ponente,
```

```
    COUNT(DISTINCT pp.id_sesion) AS sesiones_totales,
```

```
    COUNT(DISTINCT s.id_evento) AS eventos_distintos,
```

```
    -- Concatenación de roles para ver su versatilidad (ej. "Panelista, Moderador")
```

```
    STRING_AGG(DISTINCT pp.rol, ', ') AS roles_desempenados
```

```
FROM Ponente p
```

```
JOIN Participacion_Ponente pp ON p.id_ponente = pp.id_ponente
```

```
JOIN Sesion s ON pp.id_sesion = s.id_sesion
```

```
GROUP BY p.id_ponente, p.nombre
```

```
HAVING COUNT(DISTINCT pp.id_sesion) >= 1; -- Filtra solo ponentes con actividad
```